

CS2109S: Introduction to AI and Machine Learning

Lecture 4: Intro to Machine Learning & Decision Trees

3 February 2026

Capstone Project Released!



Objective: Apply the AI & ML knowledge that you have/will learn throughout the course.

- Develop an agent to play Grid Adventure V1.
 - Fully-observable, deterministic, single-agent.
 - Image observation.
- Can use any techniques: search and/or ML.
- Grading: performance on graded test cases.
- Due date: **end of semester (week 13)**.
- Released early to let you work on it gradually.

You are here



1	2	3	4	5	6	7	8	9	10	11	12	13
"Classical" AI				"Classical" ML					"Modern" ML			

Search Algorithms

- Uninformed search: BFS, DFS
- Informed search: A*
- Local Search: Hill Climbing
- Adversarial search: Minimax

"Classical" ML

- Decision Trees
- Linear/Logistic Regression
- Regularization and Kernels
- Neural networks

"Modern" ML

- Deep Learning
- Sequential Data
- Intro to unsupervised (deep) learning

Outline

- **Machine Learning**
 - Problems
 - Supervised Learning
- Decision Trees
 - Data
 - Model
 - Learning
 - Pruning
 - Real-World Data

Problems

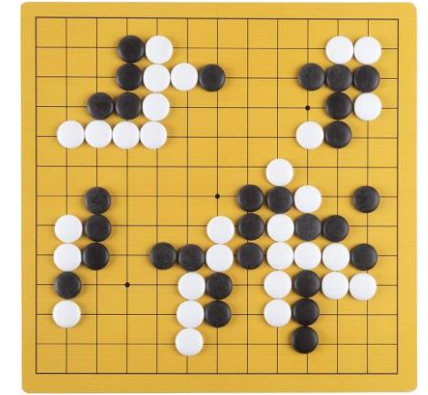
- Some problems are inherently intractable to solve, meaning that no efficient solution exists for all cases.

Examples:

- The game of Go
- Protein folding.

- Other problems are difficult to solve because formulating the rules in a way that a computer can comprehend and process is challenging. Examples:

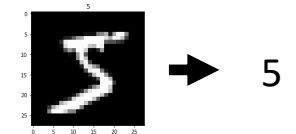
- Recognizing numbers in an image
- Answering open-ended questions
- Tasks requiring complex reasoning.



Credit: Amazon.sg

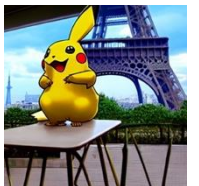
$b \approx 250, m \approx 150$ for “reasonable” games

Time complexity: $b^m = 250^{150} \approx 10^{360}$



“What is AI?” → “AI is ...”

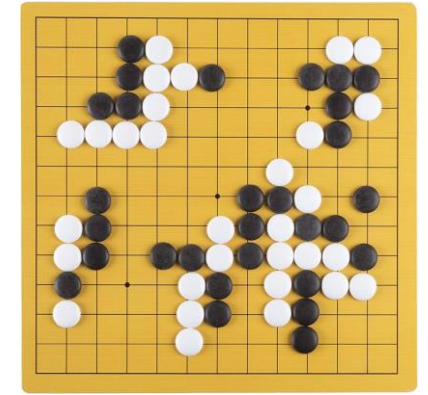
“Pikachu in the Eiffel tower” →



A New Paradigm: Learning Agent

Rather than solving the problem explicitly through search or by applying a set of rules,

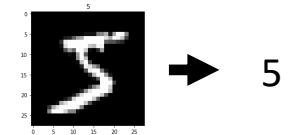
Construct an agent that learns a function to
identify patterns in the data and make decisions
or provide solutions, based on what the agent has
learned.



Credit: Amazon.sg

$b \approx 250, m \approx 150$ for “reasonable” games

Time complexity: $b^m = 250^{150} \approx 10^{360}$



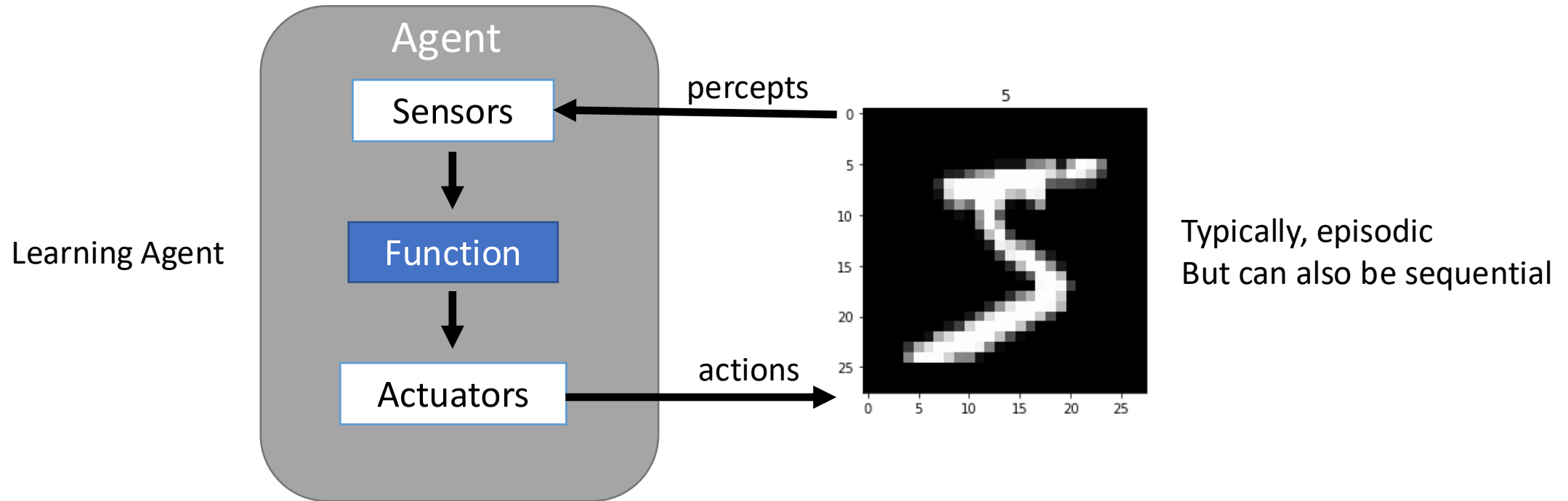
“What is AI?” → “AI is ...”

“Pikachu in the Eiffel tower” →



Designing an Agent

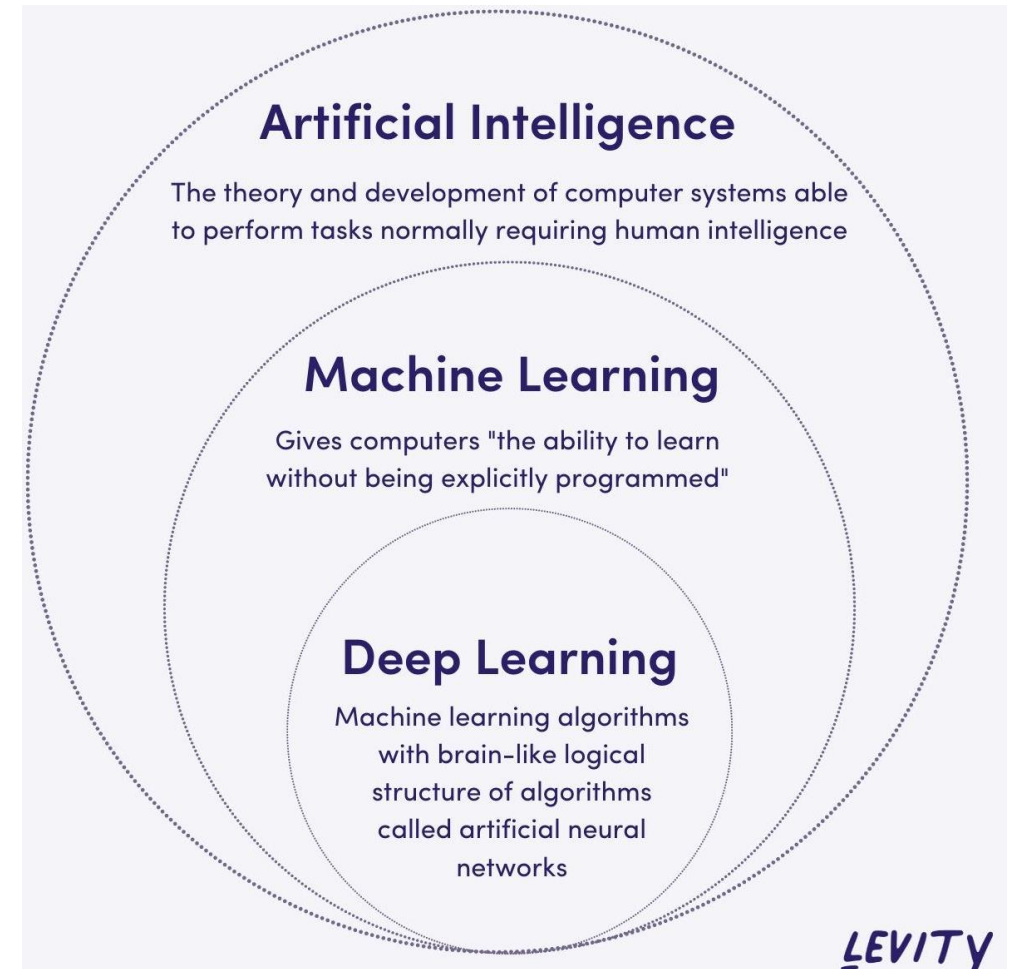
for problems where: - The function is difficult to specify, or
- Solutions are intractable to compute (in general)



Machine Learning

Machine learning (ML) is a subfield of AI that gives computers **the ability to learn without being explicitly programmed**.

It involves developing algorithms that can **learn** from and **make predictions or decisions** based on **data**.

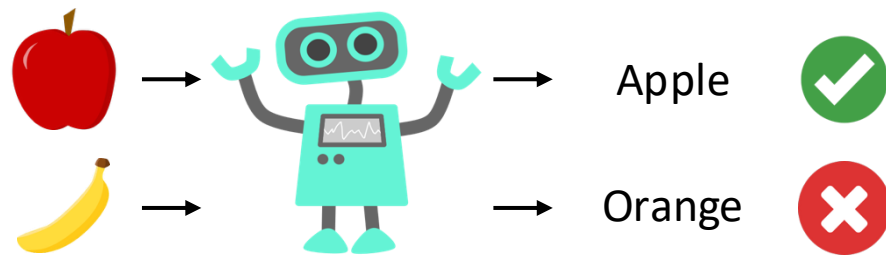


Credit: levity.ai

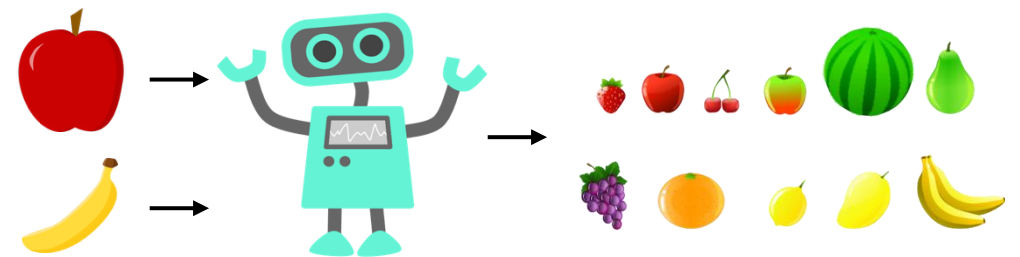
Machine Learning – Types

- **Supervised Learning:** Learn from labeled data (input-output pairs) to **learn a mapping from inputs to outputs**.
 - Example: classify whether an image contains a picture of a banana or apple
- **Unsupervised Learning:** Learn from unlabeled data (input only) to **find patterns or structure**.
 - Example: group images of fruits into 2, 3, 4, ... groups, based on features like color, shape, etc
- **Semi-supervised Learning**
- **Reinforcement Learning**

Machine Learning – Types



Supervised
"Teacher's feedback"



Unsupervised
"No feedback"

Supervised Learning

Problem-solving using supervised learning involves the following components:

- **Data:** A set of **input-output** pairs that the model learns from.
 - We assume that the data comes from the **ground truth**.
- **Model:** A function that predicts the output based on the input features.
- **Loss:** A function that is often used by a **learning algorithm** to assess the model's ability in making predictions over a set of data.

A **learning algorithm** is an algorithm which seeks to minimize the loss of a model over a dataset, called the **training dataset**.

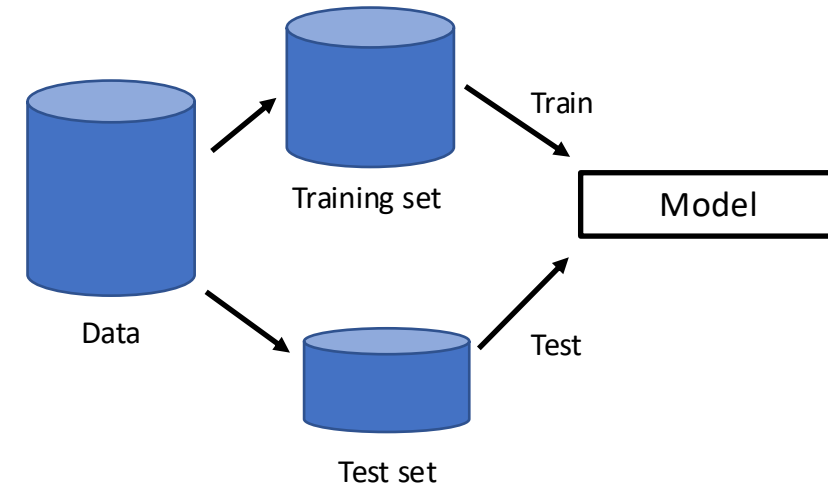
- It is used during the **training phase** of the model and results in a **trained model**.

Supervised Learning

After the training phase, the model is ready to be used.

However, before the trained model is used in the wild, it's usually tested on a set of unseen data, called **test set**.

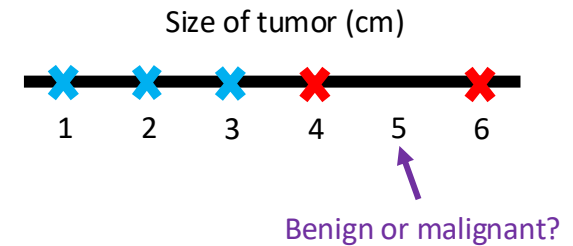
- This phase is known as the **testing / evaluation** phase.
- The performance is measured using a set of **performance measures**.
- This measures/estimates the **generalization** of the model, i.e., how well the model will perform in real-world when facing new unseen data.



Tasks

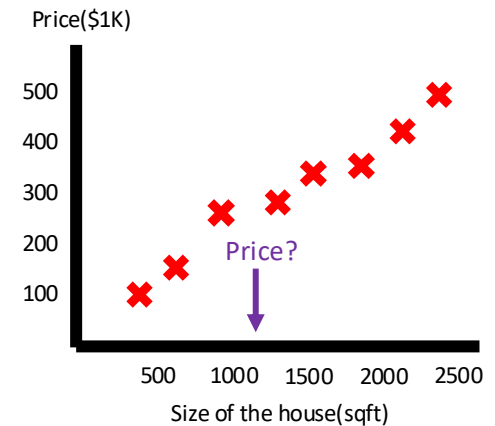
- **Classification:**

- Objective: predict a discrete label/class/category from a set of predefined classes/categories based on input features.
- The output variable (target) is a **categorical value**.
- Example: cancer prediction, spam detection, document categorization



- **Regression:**

- Objective: predict a continuous numerical value based on input features.
- The output variable (target) is a **real number**.
- Example: house price prediction, temperature prediction, sales forecasting



Data

Dataset in supervised learning is represented as a set of pairs $(x^{(i)}, y^{(i)})$, where

- $x^{(i)} \in \mathbb{R}^d$ is the input (features) vector of d length/features of the i -th data point.
 - In general, $x^{(i)}$ can be multi-dimensional array of features.
- $y^{(i)}$ is the label (target) for the i -th data point.
 - $y^{(i)} \in \mathbb{R}$ for regression
 - $y^{(i)} \in \{1, 2, \dots, K\}$ for classification with K classes

Dataset D with n examples can be represented as

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$

Model

- A **model** or **hypothesis** refers to a function that maps from inputs to outputs $h: X \rightarrow Y$ and that can be learned by a **learning algorithm**.
- We are interested in finding a model $h(x)$ that best predicts the outputs of given inputs in a dataset.
- Example (coming up later in this course):
 - Decision trees, linear/logistic models, neural networks

Performance Measure & Loss Functions

Performance measure and **loss functions** are functions that take in a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ and a model h and return a number representing how well h predicts the mapping $x^{(i)} \rightarrow y^{(i)}$.

- **Performance measure** is a metric that evaluates how well the model performs on a task. It is used during evaluation / testing.
- Technically: **loss function** is a function that quantifies the difference between the model's predictions $\hat{y}^{(i)}$ and the true outputs $y^{(i)}$. It is used to guide learning.

Performance Measure – Regression

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \mathbb{R}$, common methods of evaluating the performance of a model h are:

Mean **squared** error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

Mean **absolute** error (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n \|\hat{y}^{(i)} - y^{(i)}\|$$

Here, $h(x^{(i)}) = \hat{y}^{(i)}$.

Note: MSE and MAE can also be used as loss functions (as we will see later).

Performance Measure – Classification

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \{1, 2, \dots, K\}$, common methods of evaluating the performance of a model h are:

Accuracy (average correctness):

$$Accuracy = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\hat{y}^{(i)} = y^{(i)}}$$

Where $h(x^{(i)}) = \hat{y}^{(i)}$.

Confusion matrix, precision, recall, F1-score (next slide).

Performance Measure – Classification

No	Predicted $\hat{y}$	Actual y	
1	Cancer	Cancer	TP
2	Cancer	Cancer	
3	Benign	Cancer	FN
4	Benign	Cancer	
5	Benign	Cancer	
6	Benign	Benign	TN
7	Benign	Benign	
8	Benign	Benign	
9	Benign	Benign	FP
10	Cancer	Benign	

Confusion Matrix

		True Label	
		Cancer	Benign
Predicted Label	Cancer	2 True Positive (TP)	1 False Positive (FP)
	Benign	3 False Negative (FN)	4 True Negative (TN)

$$\text{Accuracy} = \frac{TP+TN}{TP+FN+FP+TN}$$

Precision

$$P = TP / (TP+FP)$$

Recall

$$R = TP / (TP+FN)$$

F1 Score
(Harmonic mean between precision and recall)

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$$

If false positives are costly
e.g., spam prediction (removed emails)

If false negatives are costly
e.g., cancer prediction (missed cancer)

Balance precision & recall

[True/False] [Positive/Negative]

“The prediction is [Positive/Negative] and it is [True/False]”

Learning Algorithms

- A **learning algorithm** A takes in a training set D_{train} , consisting of pairs $(x^{(i)}, y^{(i)})$, and seeks to find a model h that approximates the true relationship between inputs and outputs, i.e., $h(x^{(i)}) \approx y^{(i)}$.
- Most (not all) algorithms utilize a loss function to help guide the finding/optimization of the model.
- A learning algorithm is usually quite general and can work for different data, model, and loss.
- Example (coming up later in this course):
 - Decision tree learning, gradient descent

Supervised Learning Diagram

Training

Training Set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(n_{train})}, y^{(n_{train})})\}$



Learning Algorithm A ←····· Loss Function
Find $h(x)$ that best fits the training set



Model $h(x)$



Testing

Test Set
 $\{(x^{(1)}, y^{(1)}), \dots, (x^{(n_{test})}, y^{(n_{test})})\}$

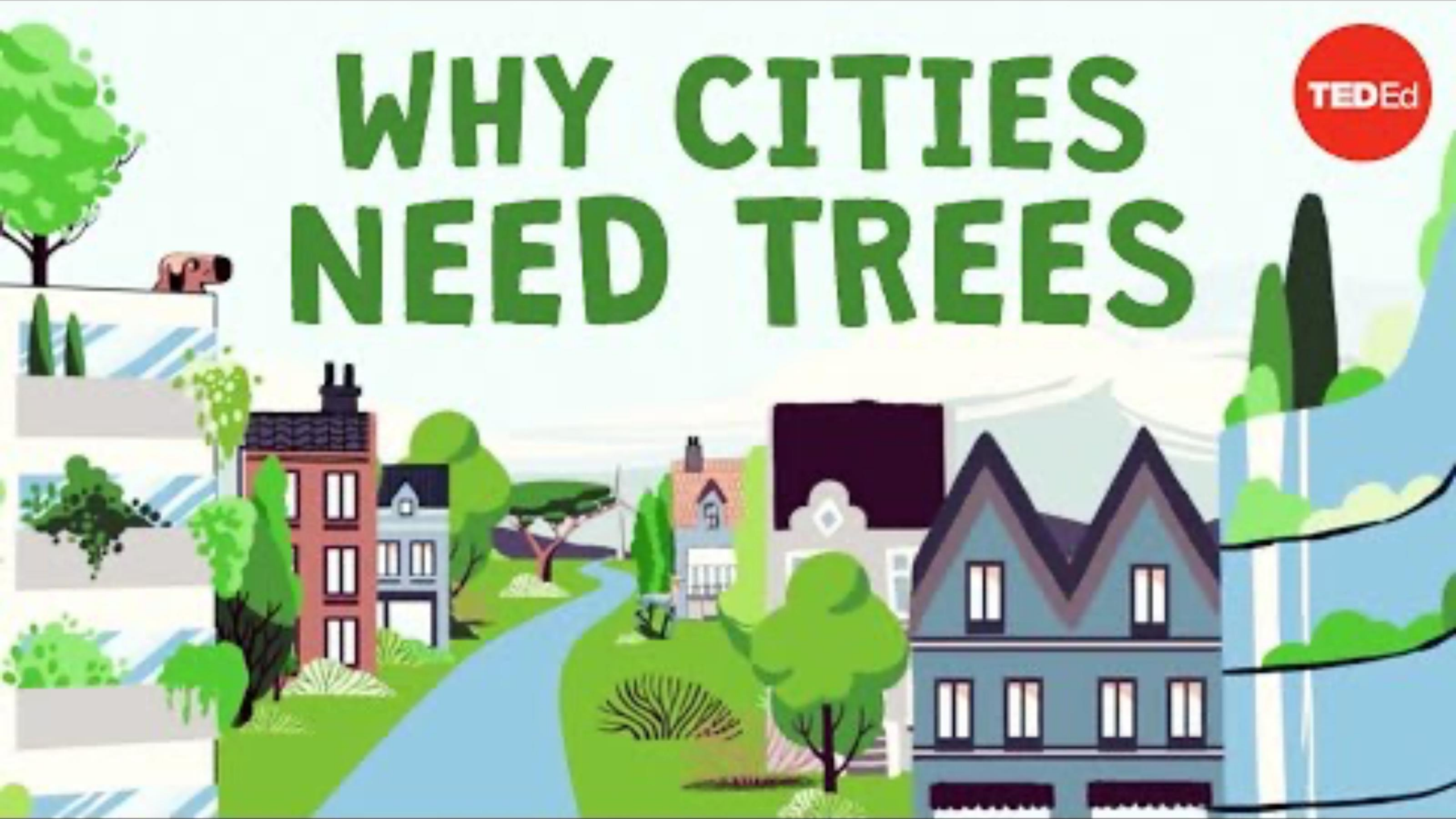


Performance Measure
Check $h(x)$ fits in test set
(Generalization)

Break

WHY CITIES NEED TREES

TEDEd



Outline

- Machine Learning
 - Problems
 - Supervised Learning
- **Decision Trees**
 - Data
 - Model
 - Learning
 - Pruning
 - Real-World Data

Example: Hiring Decision

Given a candidate for a job. Would you hire them?

Hire?
?

For the hiring decision, it makes sense to look at attributes of the candidate.

Experience	Interview	Skills	Hire?
Junior	Good	High	?

Example: Hiring Decision

For the hiring decision, it makes sense to look at data of previous hires.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	No

What is the hiring decision, given the data of previous hires?

Data

Suppose:

- We are given n data points.
- Each data point consists of **features** and a **target** variable.
- The features are described by a feature vector or row with d number of **categorical** features (attributes).
- The target is $\{1, 2, \dots, K\}$, where K is the number of classes.

Data – Math

Suppose:

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(\textcolor{red}{n})}, y^{(\textcolor{red}{n})})\},$$

where for all $i \in \{1, \dots, \textcolor{red}{n}\}$.

- Features: $x^{(i)} \in \{c_1, c_2, \dots, c_m\}^{\textcolor{blue}{d}}$ for some categories $c_1, c_2, \dots, c_m$.
- Targets: $y^{(i)} \in \{1, 2, \dots, K\}$ for K classes.

Task

Suppose we are given another data point $x \in \{c_1, c_2, \dots, c_m\}^d$ and no target. Based on the dataset, find a function that predicts the target $y \in \{1, 2, \dots, K\}$ for that x .

This is a classification task.

Example: Hiring Decision (Formalized)

Given a candidate x with features experience (x_1), interview result (x_2), and skills (x_3),
What is the output (y)?

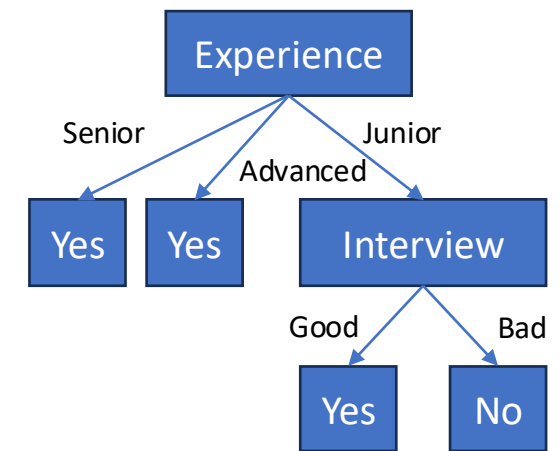
	Experience (x_1)	Interview (x_2)	Skills (x_3)	Hire? (y)
$x^{(1)}$	Senior	Good	High	Yes
$x^{(2)}$	Advanced	Bad	High	Yes
$x^{(3)}$	Junior	Good	Mid	Yes
$x^{(4)}$	Junior	Bad	High	No
$x^{(5)}$	Junior	Bad	Mid	No

Decision Tree Model

A **decision tree** is a model that implements a function that takes in an input of a vector of **categorical** attribute values $x \in \{c_1, c_2, \dots, c_m\}^d$ and output a decision or prediction $y \in \{1, 2, \dots, K\}$.

A decision tree is a tree structure where:

- **Internal nodes** represent tests on features
- **Branches** correspond to outcomes of the tests
- **Leaf nodes** assign predictions

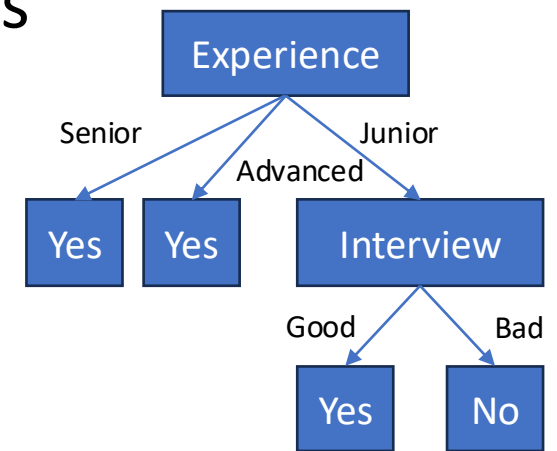


Decision Tree Model – Prediction

A decision is reached by sequentially checking the **attribute value at each node**, starting from the root and following the branches corresponding to the outcomes of these tests, until a leaf node is encountered.

Basically, running through a nested if-else.

Given a candidate with **Junior** experience, **Good** interview, and **High** skills, would you hire them?



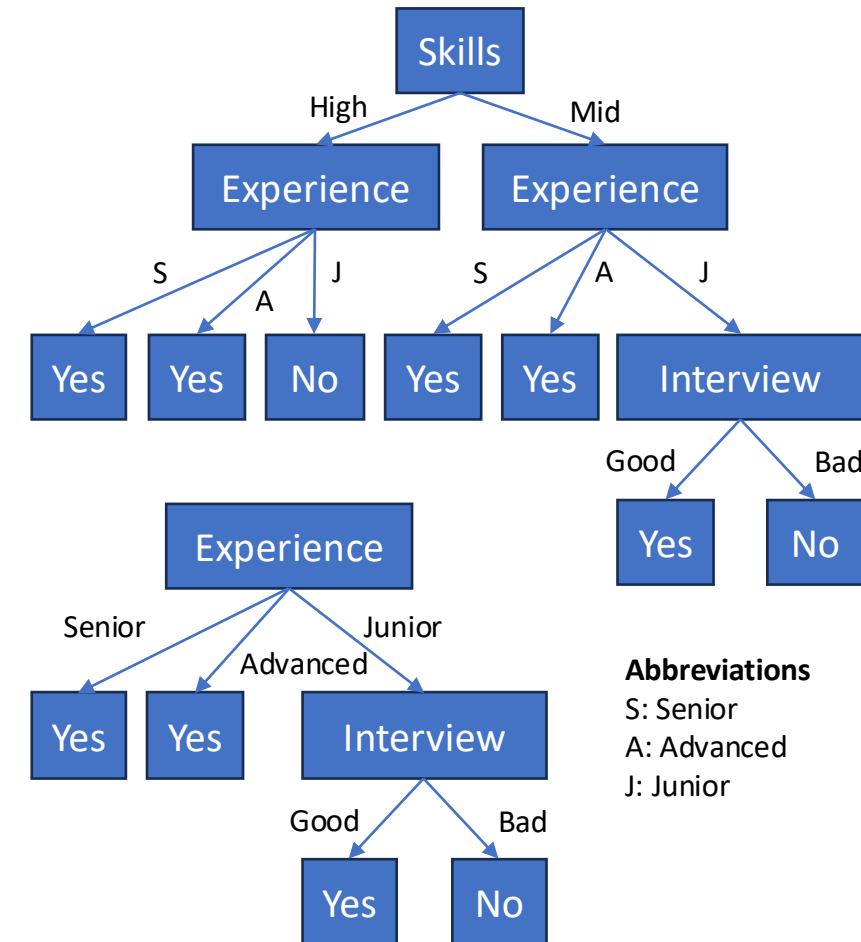
Optimal Decision Tree Model

There generally exist multiple (in fact, potentially many) **distinct decision tree models** that are all consistent with the training data—that is, they all classify every training example correctly.

Prefer a "compact" decision tree: the decision tree has the smallest possible number of nodes. Also called the **optimal decision tree**.

A "compact" decision tree is less likely to be consistent with training examples due to chance.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	No



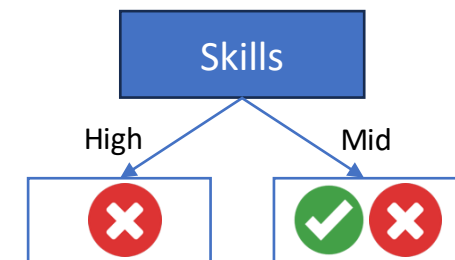
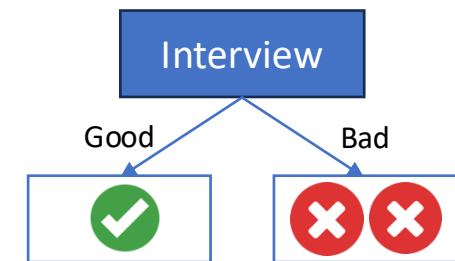
Learning a Decision Tree Model

Theorem: The problem of finding an **optimal decision tree** that is consistent with all the training examples is **NP-Hard**.

Implication: make tractable by greedy heuristics.

- Make locally optimal choices at each step by selecting the feature that yields the greatest immediate **information gain**, to partition the data as decisively as possible.

No	Interview	Skills	Hire?
3	Good	Mid	Yes
4	Bad	High	No
5	Bad	Mid	No



Background: Entropy

Entropy quantifies the uncertainty or randomness of a random variable.

Let Y be a random variable with outcomes values $y_1, \dots, y_k$ and probabilities $P(y_1), \dots, P(y_k)$. Then, the entropy is defined as:

$$H(Y) = H(P(y_1), \dots, P(y_k)) = - \sum_{i=1}^k P(y_i) \log_2 P(y_i)$$

If the outcomes are decisions, entropy measures the level of **uncertainty** (or **indecisiveness**) about a decision.

Background: Entropy for Binary Outcomes

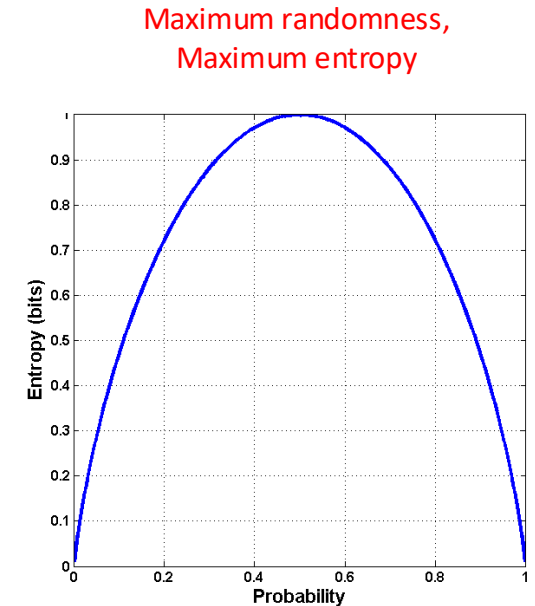
For a dataset which contains p positive and n negative outcomes.

Probabilities:

$$P(\text{pos}) = \frac{p}{p+n} \text{ and } P(\text{neg}) = \frac{n}{p+n}$$

Entropy:

$$\begin{aligned} H(Y) &= H(P(\text{pos}), P(\text{neg})) = - \sum_{y_i \in \{\text{pos}, \text{neg}\}} P(y_i) \log_2 P(y_i) \\ &= - \left(\frac{p}{p+n} \log_2 \frac{p}{p+n} \right) - \left(\frac{n}{p+n} \log_2 \frac{n}{p+n} \right) \end{aligned}$$



Background: Entropy – Example

Hire? (y)
Yes
Yes
Yes
No
No

Probabilities:

$$P(\text{pos}) = \frac{p}{p+n} = \frac{3}{5}$$

$$P(\text{neg}) = \frac{n}{p+n} = \frac{2}{5}$$

Entropy:

$$H(Y) = H(P(\text{pos}), P(\text{neg}))$$

$$= H\left(\frac{3}{5}, \frac{2}{5}\right)$$

$$= -\left(\frac{3}{5} \log_2 \frac{3}{5}\right) - \left(\frac{2}{5} \log_2 \frac{2}{5}\right)$$

$$= 0.971$$

Background: Specific Conditional Entropy

Specific conditional entropy $H(Y|X = x)$ is the entropy of a random variable Y conditioned on the discrete random variable X taking a certain **fixed** value x .

For random variable Y with k possible outcomes, it is defined as:

$$H(Y|X = x) = H(P(y_1|x), \dots, P(y_k|x))$$

Experience	Hire? (y)
Senior	Yes
Advanced	Yes
Junior	Yes
Junior	No
Junior	No

For example, the specific conditional entropy for hiring decision given that “Experience” is taking a value “Junior” is:

$$\begin{aligned}
 H(Y|Experience = \text{Junior}) &= H\left(\frac{1}{3}, \frac{2}{3}\right) \\
 &= -\left(\frac{1}{3} \log_2 \frac{1}{3}\right) - \left(\frac{2}{3} \log_2 \frac{2}{3}\right) \\
 &= 0.918
 \end{aligned}$$

Background: Conditional Entropy

Conditional entropy $H(Y|X)$ quantifies the amount of uncertainty (or entropy) that remains about a random variable Y when another random variable X is known.

The formal definition is:

$$H(Y|X) = \sum_{x \in X} P(X = x) H(Y|X = x)$$

If the possible outcomes correspond to decisions, conditional entropy represents the level of **uncertainty** (or **indecisiveness**) about a decision after another variable has been observed or chosen.

Background: Conditional Entropy – Example

Experience	Hire? (y)
Senior	Yes
Advanced	Yes
Junior	Yes
Junior	No
Junior	No

Abbreviations

S: Senior

A: Advanced

J: Junior

$$\begin{aligned}
 H(Y|Experience) &= \sum_{x \in \{S, A, J\}} P(Experience = x) H(Y|Experience = x) \\
 &= \frac{1}{5} \times 0 + \frac{1}{5} \times 0 + \frac{3}{5} \times 0.918 = 0.551
 \end{aligned}$$

$$P(Experience = S) = \frac{1}{5}$$

$$P(Experience = A) = \frac{1}{5}$$

$$P(Experience = J) = \frac{3}{5}$$

$$H(Y|Experience = S) = H(1) = 0$$

$$H(Y|Experience = A) = H(1) = 0$$

$$H(Y|Experience = J) = H\left(\frac{1}{3}, \frac{2}{3}\right) = -\left(\frac{1}{3} \log_2 \frac{1}{3}\right) - \left(\frac{2}{3} \log_2 \frac{2}{3}\right) = 0.918$$

Information Gain

Information gain is a measure of the reduction in **uncertainty** or **indecisiveness** about a particular decision after knowing the value of a specific variable.

The formula for information gain is:

$$IG(Y; X) = \underbrace{H(Y)}_{\text{Uncertainty before}} - \underbrace{H(Y|X)}_{\text{Uncertainty after selecting a variable}}$$

Example (using data from the previous slide):

$$IG(Y; Experience) = H(Y) - H(Y|Experience) = 0.971 - 0.551 = 0.42$$

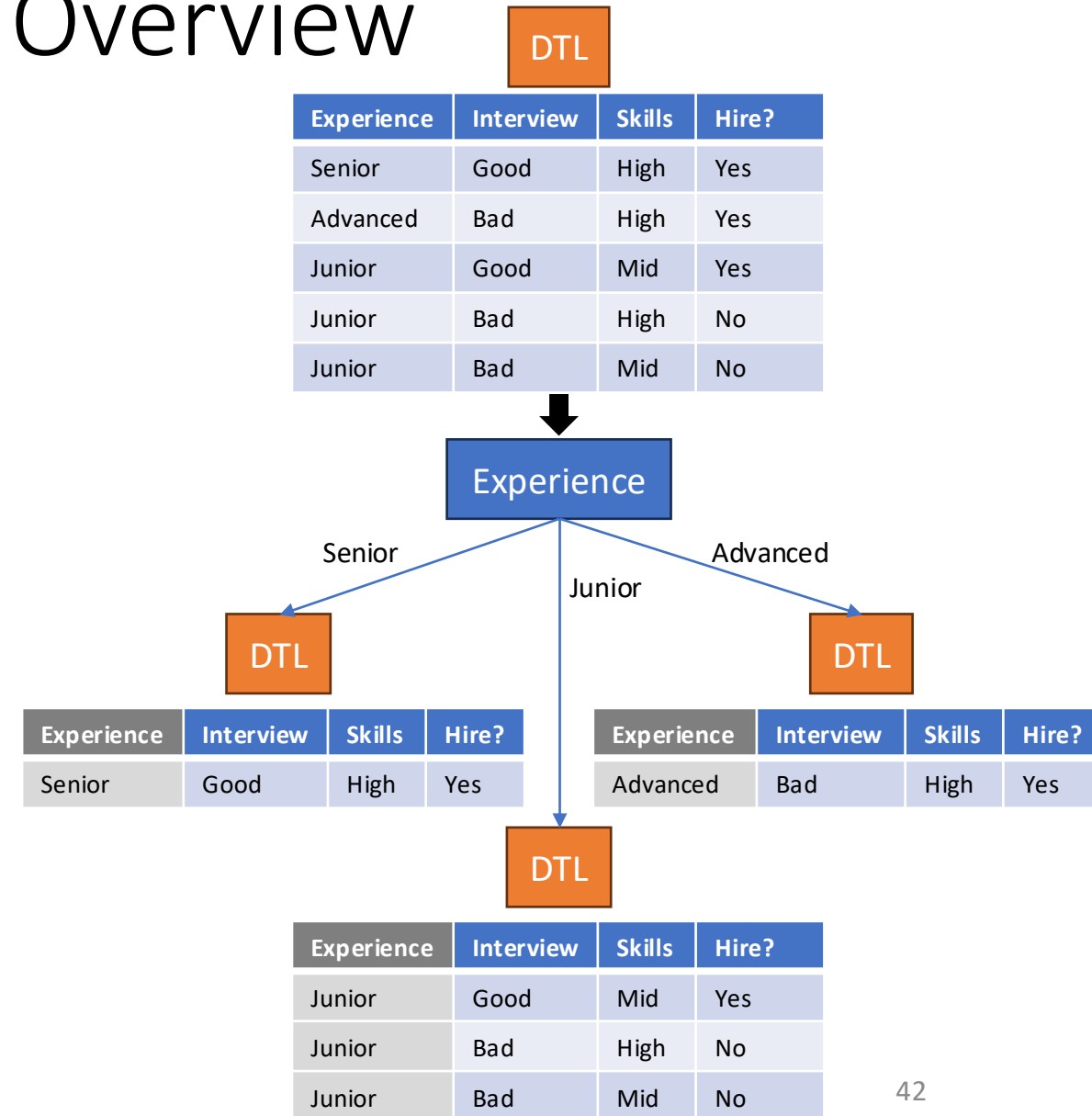
Decision Tree Learning – Overview

Top-down, greedy, recursive algorithm.

- **Top-down:** Build the decision tree from the root.
- **Greedy:** At each step, choose the locally optimal attribute to split, such as based on immediate information gain, without considering future splits.
- **Recursive:** For each value of a chosen attribute, use DTL again on corresponding subset of data

Stopping criteria:

- No more data or attributes, or
- All data predicts the same class.



Decision Tree Learning – Functions

- **choose_attribute(attributes, rows)**
 - Accepts a set of **attributes** (table columns) in the data and the table **rows** containing the data points.
 - Compute the **information gain** of each attribute in **attributes**.
 - Returns the attribute that has the highest information gain.
- **majority_decision(rows)**
 - Accepts the table **rows** containing the data points.
 - Returns the most common decision in the data points.

Decision Tree Learning – Base Cases

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
    ...
```

1

Experience	Interview	Skills	Hire?
------------	-----------	--------	-------

 No data available → Default

2

Interview	Hire?
Bad	No
Bad	No

 Same decision for all rows → “No”

3

Hire?
Yes
Yes
Yes
No
No

 Ran out of attributes
3 “Yes”, 2 “No” → “Yes”

When do the base cases happen?

- 1 No training data for a particular attribute value. DTL called with no data.
- 2 Most common case. After splitting data according to values.
- 3 Inconsistent data. Noisy data. Insufficient attributes to describe data.

Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    > best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

Decision entropy:

$$H(Y) = H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

DTL

Conditional entropy:

$$H(Y|Experience) = \frac{1}{5}H(1) + \frac{1}{5}H(1) + \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.551$$

$$H(Y|Interview) = \frac{2}{5}H(1) + \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.551$$

$$H(Y|Skills) = \frac{3}{5}H\left(\frac{1}{3}, \frac{2}{3}\right) + \frac{2}{5}H\left(\frac{1}{2}, \frac{1}{2}\right) = 0.951$$

Information gain:

$$IG(Y; Experience) = H(Y) - H(Y|Experience) = 0.42$$

$$IG(Y; Interview) = H(Y) - H(Y|Interview) = 0.42$$

$$IG(Y; Skills) = H(Y) - H(Y|Skills) = 0.02$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute

    1 for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        2 subtreei = DTL(subrowsi,
                           attributes - best_attribute,
                           majority_decision(subrowsi))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

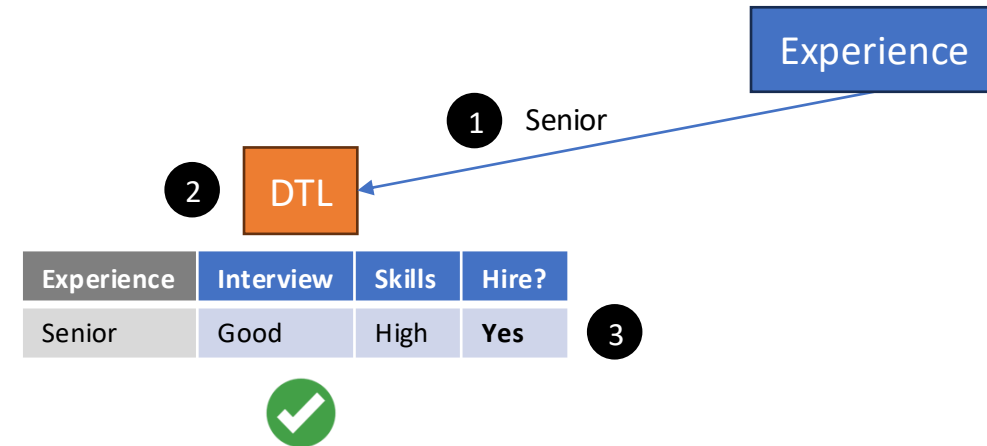
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
  
    best_attribute = choose_attribute(attributes, rows)  
    tree = a new decision tree with root best_attribute  
    for valuei of best_attribute:  
        subrowsi = {rows with best_attribute = valuei}  
        subtreei = DTL(subrowsi,  
                        attributes - best_attribute,  
                        majority_decision(rows))  
    > add subtreei as a branch to tree with label valuei  
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

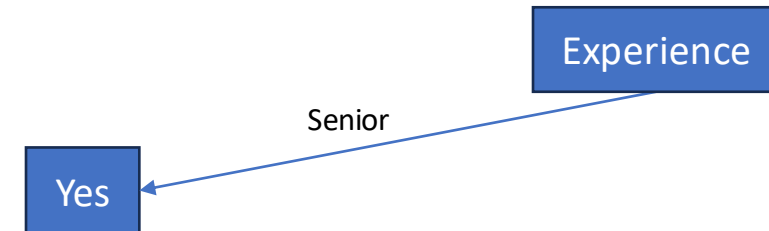
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

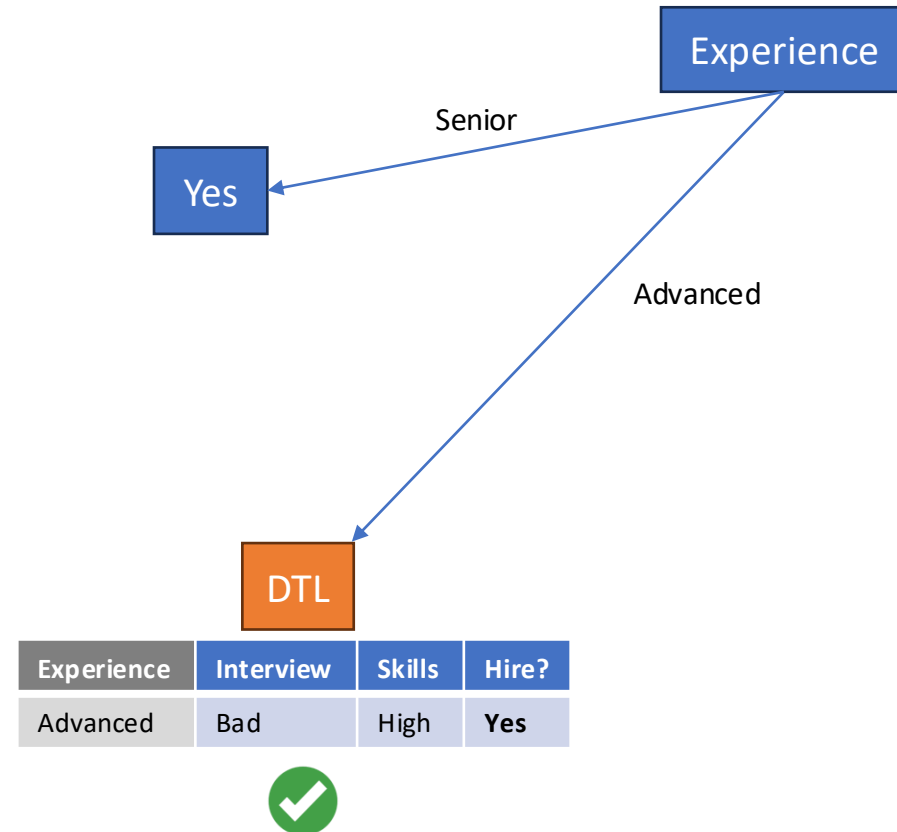
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
  
    best_attribute = choose_attribute(attributes, rows)  
    tree = a new decision tree with root best_attribute  
    for valuei of best_attribute:  
        subrowsi = {rows with best_attribute = valuei}  
        subtreei = DTL(subrowsi,  
                        attributes - best_attribute,  
                        majority_decision(rows))  
        add subtreei as a branch to tree with label valuei  
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

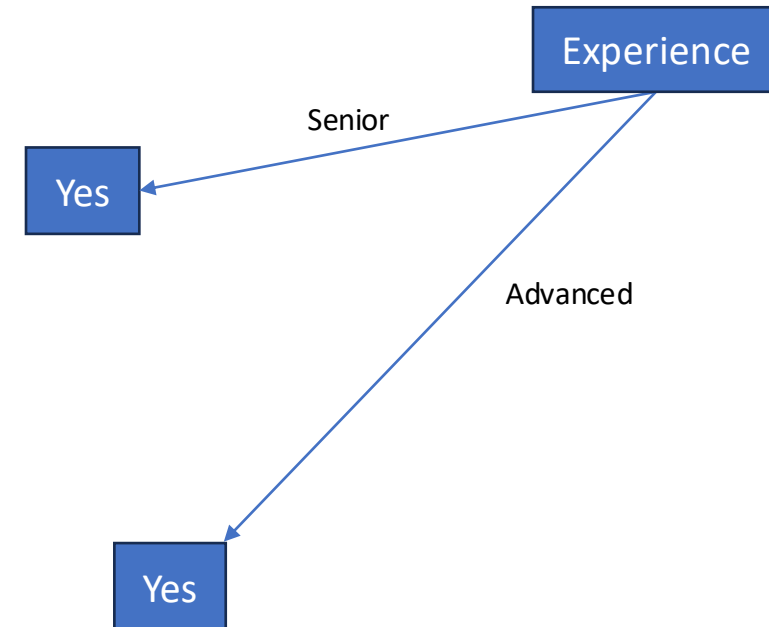
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

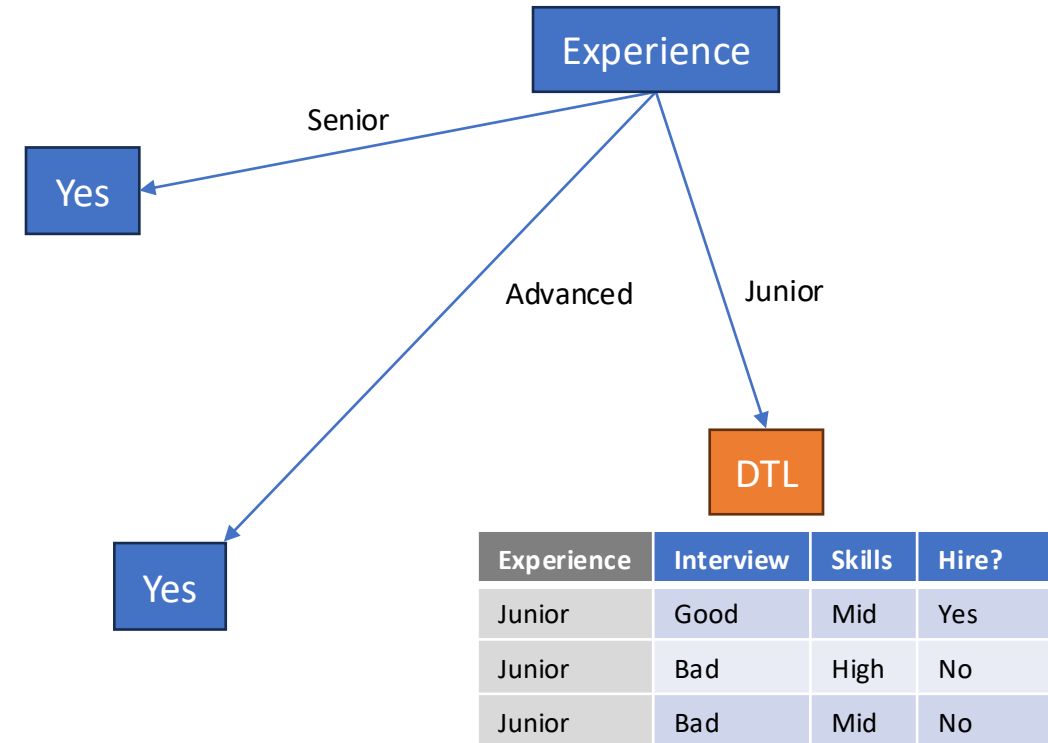
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
    if rows is empty: return default
    if rows have the same classification:
        return classification
    if attributes is empty:
        return majority_decision(rows)

    best_attribute = choose_attribute(attributes, rows)
    tree = a new decision tree with root best_attribute
    for valuei of best_attribute:
        subrowsi = {rows with best_attribute = valuei}
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
        add subtreei as a branch to tree with label valuei
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

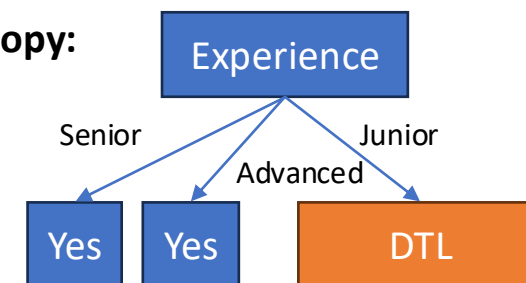
$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$

Decision (specific conditional) entropy:

$$H(Y|Experience = J)$$

$$= H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$



Conditional entropy:

$$H(Y|Experience = J, Interview)$$

$$= \frac{1}{3}H(1) + \frac{2}{3}H(1) = 0$$

$$H(Y|Experience = J, Skills)$$

$$= \frac{2}{3}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{1}{3}H(1) = 0.666$$

Experience	Interview	Skills	Hire?
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Abbreviation

J: Junior

Information gain:

$$IG(Y|Experience = J; Interview) = 0.918$$

$$IG(Y|Experience = J; Skills) = 0.252$$



Decision Tree Learning

```
def DTL(rows, attributes, default):
```

```
    if rows is empty: return default
```

```
    if rows have the same classification:
```

```
        return classification
```

```
    if attributes is empty:
```

```
        return majority_decision(rows)
```

```
    best_attribute = choose_attribute(attributes, rows)
```

```
    tree = a new decision tree with root best_attribute
```

```
    for valuei of best_attribute:
```

```
        subrowsi = {rows with best_attribute = valuei}
```

```
        subtreei = DTL(subrowsi,
                        attributes - best_attribute,
                        majority_decision(rows))
```

```
        add subtreei as a branch to tree with label valuei
```

```
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

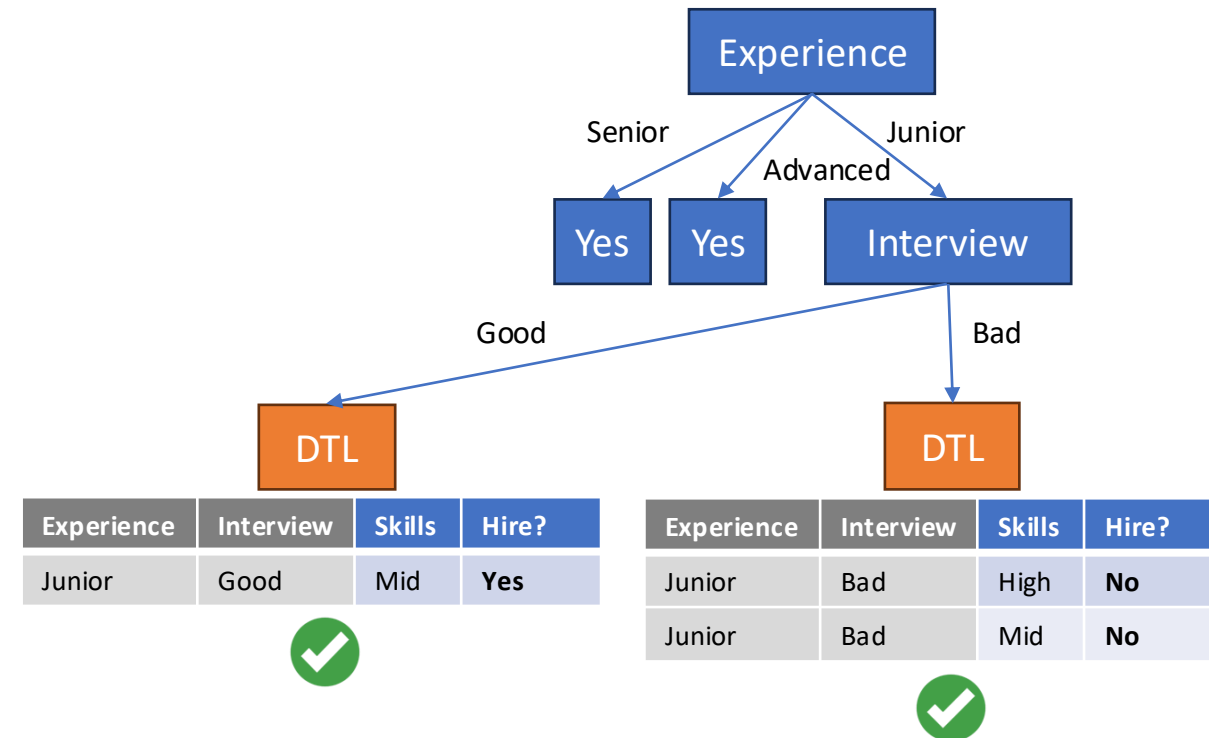
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Decision Tree Learning

```
def DTL(rows, attributes, default):  
    if rows is empty: return default  
    if rows have the same classification:  
        return classification  
    if attributes is empty:  
        return majority_decision(rows)  
  
    best_attribute = choose_attribute(attributes, rows)  
    tree = a new decision tree with root best_attribute  
    for valuei of best_attribute:  
        subrowsi = {rows with best_attribute = valuei}  
        subtreei = DTL(subrowsi,  
                        attributes - best_attribute,  
                        majority_decision(rows))  
        add subtreei as a branch to tree with label valuei  
    return tree
```

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

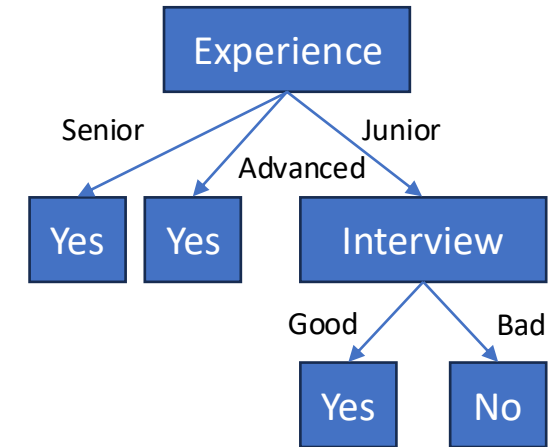
Common entropies:

$$H(1) = 0$$

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = 1$$

$$H\left(\frac{1}{3}, \frac{2}{3}\right) = 0.918$$

$$H\left(\frac{3}{5}, \frac{2}{5}\right) = 0.971$$



Poll Everywhere

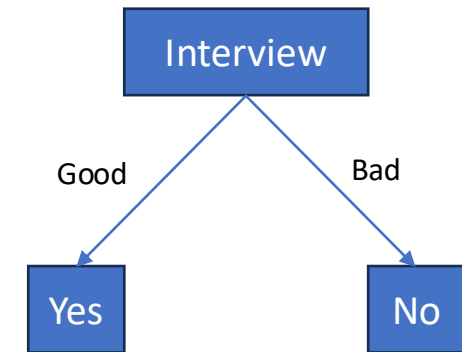
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Junior	Bad	Low	No
Senior	Good	High	Yes
Advanced	Bad	Mid	No
Junior	Good	Mid	Yes
Advanced	Bad	High	No

Suppose you learn a decision tree model using decision tree learning algorithm and the data provided.

What is the root node of the decision tree model?

Poll Everywhere

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Junior	Bad	Low	No
Senior	Good	High	Yes
Advanced	Bad	Mid	No
Junior	Good	Mid	Yes
Advanced	Bad	High	No



Suppose you learn a decision tree model using decision tree learning algorithm and the data provided.

What is the root node of the decision tree model?

Decision Tree – Analysis

Theorem (Representational Completeness of Decision Tree):

For any finite set of consistent examples with discrete (categorical) features and a finite set of label classes, there exists a **decision tree that is consistent** with (i.e., **perfectly classifies**) the examples.

The Problem with Decision Tree

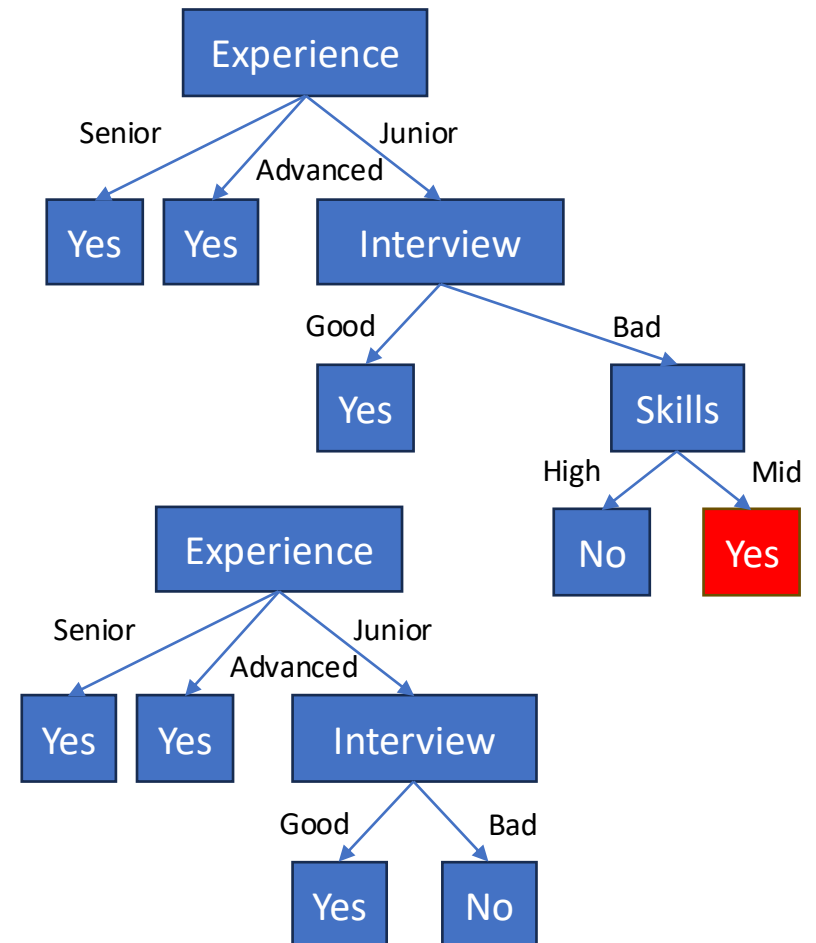
Representational completeness means that a decision tree can, in principle, fit any finite set of labeled training data exactly, including:

- Patterns that represent real, generalizable relationships
- Any **noise or outliers**.

The latter may cause the learned decision tree to be **overly (and unnecessarily) complex** as it tries to capture unnecessary information in the training data.

This phenomenon is called **overfitting**.

No	Experience	Interview	Skills	Hire?
1	Senior	Good	High	Yes
2	Advanced	Bad	High	Yes
3	Junior	Good	Mid	Yes
4	Junior	Bad	High	No
5	Junior	Bad	Mid	Yes



Pruning

Pruning refers to the process of **removing parts of the tree**. The goal is to produce a simpler, more robust tree that generalizes better to unseen data by **limiting the representational capability** of a decision tree.

Common pre-pruning (early stopping) methods:

- **Max depth**: limits the maximum depth of the decision tree. The depth of a tree is the length of the longest path from the root node to a leaf node.
- **Min-sample leaves**: set a minimum threshold for the number of samples required to be in a leaf node. If a leaf node has fewer than this minimum number of samples, it should be pruned.

Pruning can also be done after the tree is fully grown, this is called post-pruning methods, which is beyond the scope of this course.

Pruning: Max-Depth

Max-depth = 1

Depth 0

DTL

Depth 1

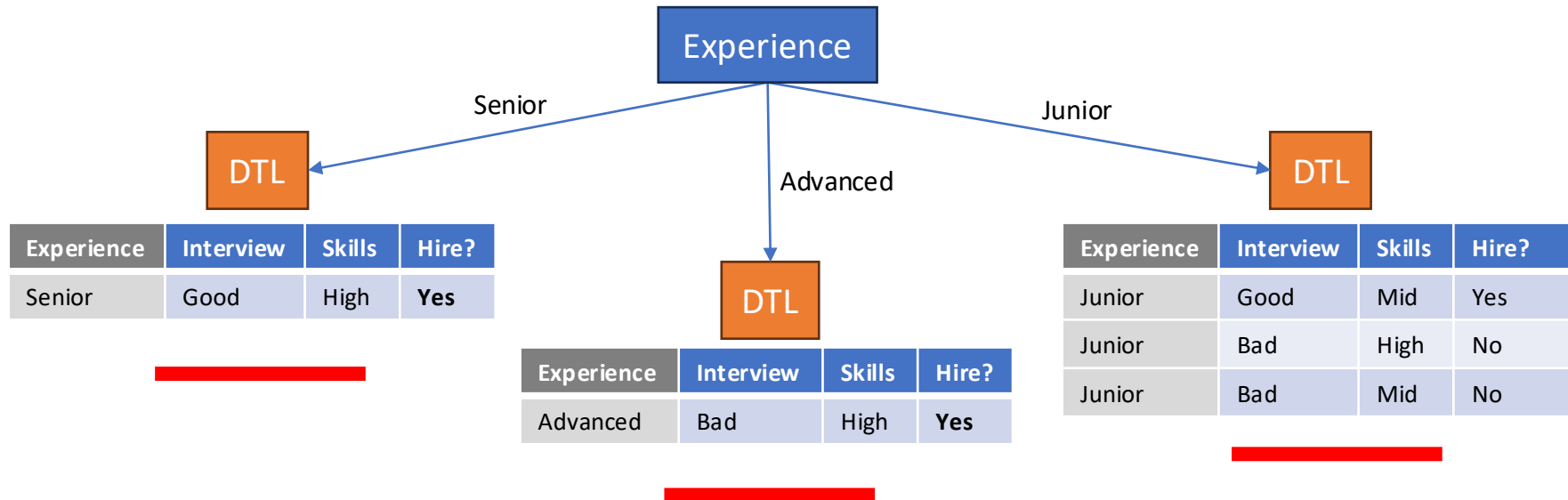
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Max-Depth

Max-depth = 1

Depth 0

Depth 1



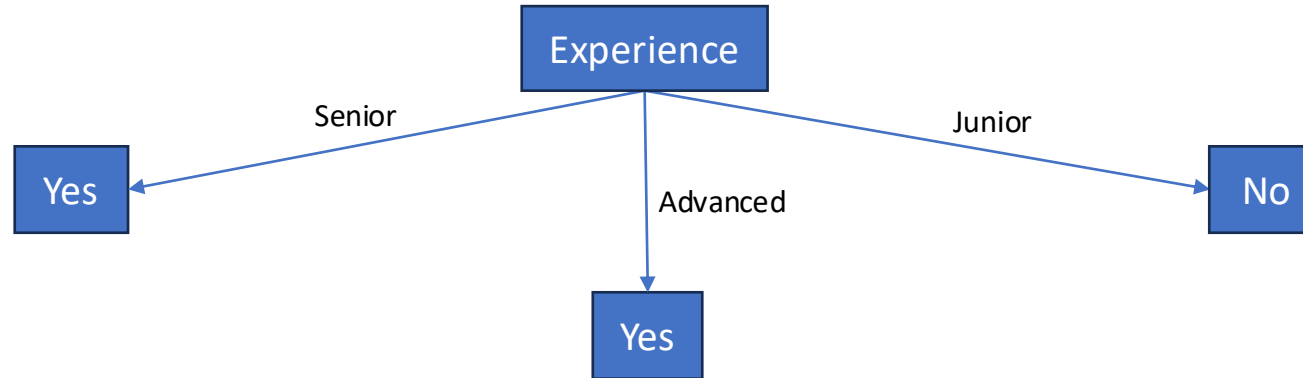
Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Max-Depth

Max-depth = 1

Depth 0

Depth 1



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Advanced	Bad	High	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

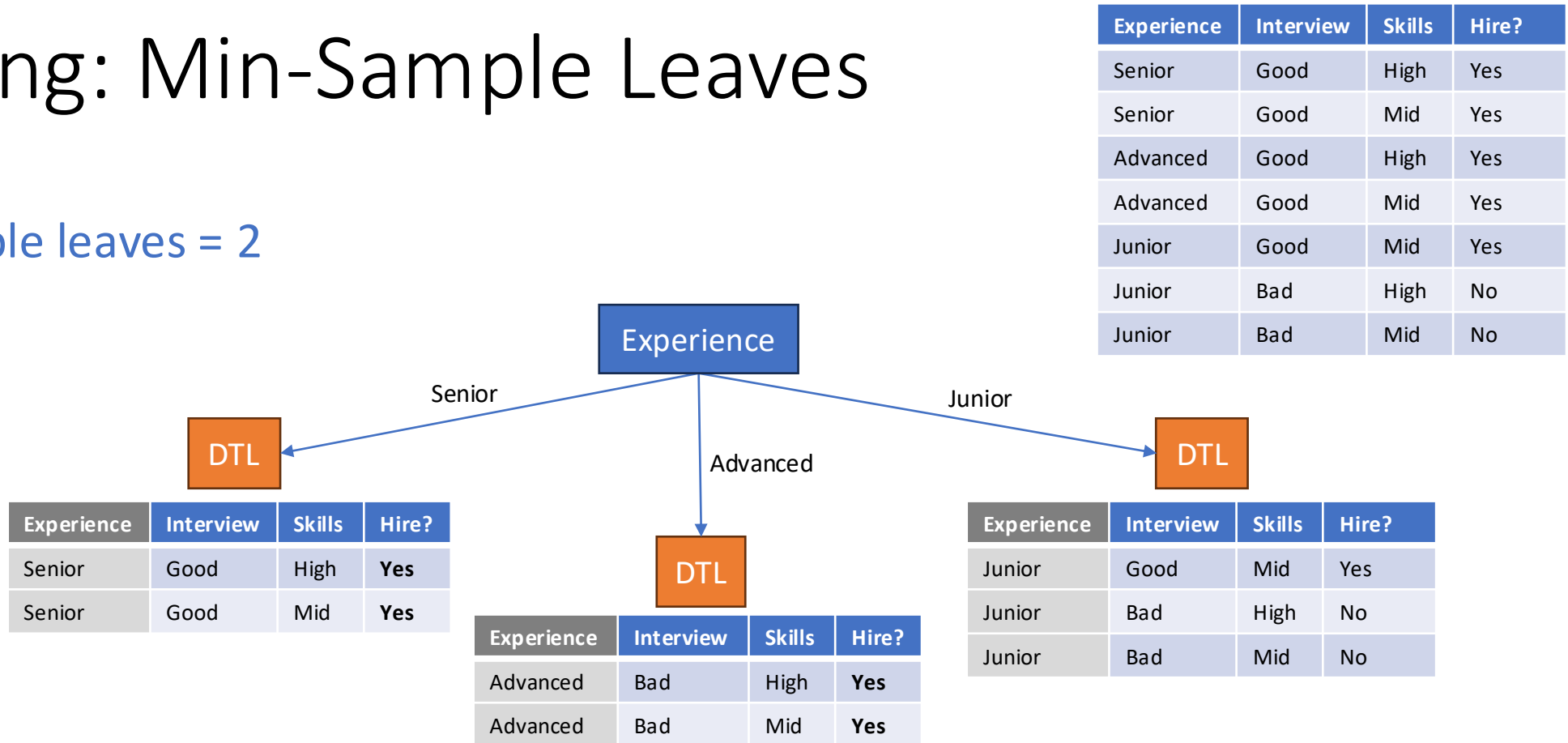
Min-sample leaves = 2

DTL

Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

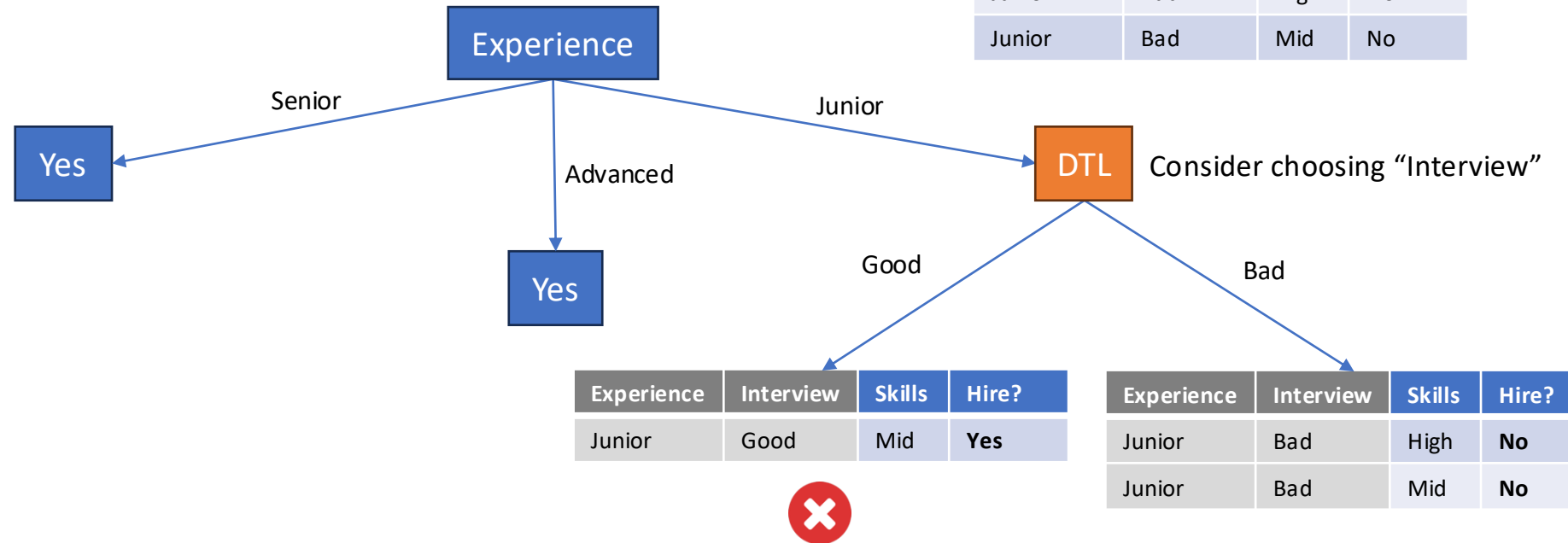
Pruning: Min-Sample Leaves

Min-sample leaves = 2



Pruning: Min-Sample Leaves

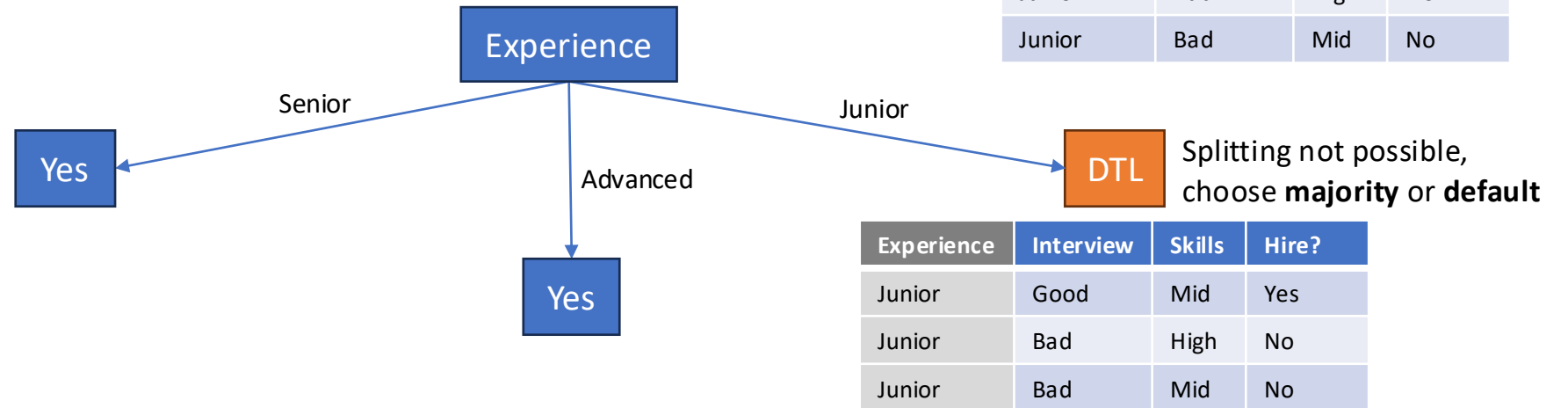
Min-sample leaves = 2



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

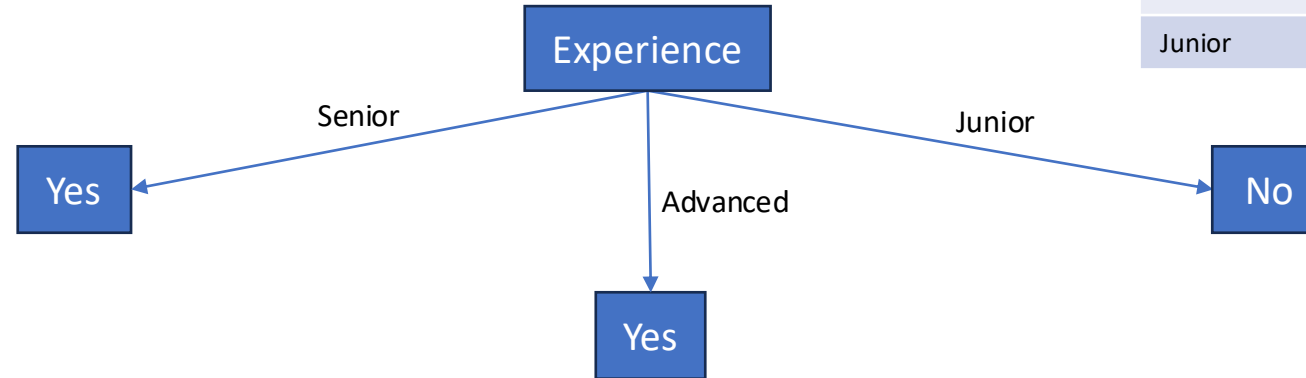
Min-sample leaves = 2



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Pruning: Min-Sample Leaves

Min-sample leaves = 2



Experience	Interview	Skills	Hire?
Senior	Good	High	Yes
Senior	Good	Mid	Yes
Advanced	Good	High	Yes
Advanced	Good	Mid	Yes
Junior	Good	Mid	Yes
Junior	Bad	High	No
Junior	Bad	Mid	No

Real-World Data

Real-world data is often messy and **may contain attributes or values that are incompatible** with decision tree models, such as attributes with continuous values or missing values. In such cases, [preprocessing](#) the data is essential.

- **For attributes with continuous values:**

- Partition the values into a discrete set of intervals.
- Example:
Experience (viewed as a real-valued time period) can be partitioned into Junior (<1 year), Advanced (1-3 years), Senior (> 3 years).

- **For attributes with missing values:**

- Assign the most common value of the attribute
- Assign the most common value of the attribute with the same output
- Assign probability to each possible value and sample
- Drop the attribute
- Drop the rows
- ...

Resources

Scikit Learn

- Decision Trees
 - <https://scikit-learn.org/stable/modules/tree.html>
- Ensembles
 - <https://scikit-learn.org/stable/modules/ensemble.html>

Summary

- Machine Learning
 - Designing an agent for intractable problems or problems with difficult to specify rules.
 - Machine learning: ability to learn without explicitly being programmed
 - Types: supervised, unsupervised, semi-supervised, reinforcement
 - Supervised Learning: learning with labels
 - Data, Model, Loss
 - Regression, Classification
 - Performance measures:
 - Classification: Accuracy, Precision, Recall, F1 score.
 - Regression: MSE, MAE.
- Decision Trees
 - Decision Tree Model
 - Entropy, conditional entropy, information gain
 - Decision tree learning algorithm (DTL): Greedy maximization of the information gain.
 - Pruning: max depth, min-sample leaves
 - Real-World Data is messy, need preprocessing

Further Reading (Optional)

- Representation Completeness of Decision Tree Model
 - Mitchell, T. M. (1997). *Machine Learning*, Chapter 3. - Quinlan, J. R. (1986). "Induction of decision trees." *Machine Learning*, 1(1), 81–106.
- Optimal Decision Tree
 - Hyafil, L., & Rivest, R. L. (1976). "Constructing optimal binary decision trees is NP-complete." *Information Processing Letters*, 5(1), 15–17. - Mitchell, T. M. (1997). *Machine Learning*, Chapter 3.
- Post-pruning methods (TESL, Chapter 9)
- Gradient boosted tree (TESL, Chapter 10)
- Random forests (TESL, Chapter 15)

Coming Up Next Week

- Linear Regression
- Learning Algorithms:
 - Gradient Descent
 - Normal Equation

To Do

- **Lecture Training 4**
 - +550 EXP
 - +100 Early bird bonus
- **Problem Set 2**
 - Will be released (hopefully) later today!

Exercise 1

We have 4 patients of group A and 6 patients of group B

What is your estimate of the true probability for group A and group B?

- a. 1, 0
- b. 0.5, 0.5
- c. 0.4, 0.6
- d. 0, 1

The intent is to make you think about the sample probability versus the true probability. (This step is implicit in Slide 35 and 38)

Exercise 2

Experience	Hire?
Senior	Yes
Advanced	Yes
Junior	Yes
Junior	No
Junior	No

Mind bender: Let's swap the role of "Experience" and "Hire" and look at the conditional entropy.

Select the conditional entropy for Experience given that the Hiring decision is Yes.

1.0

2.0.918

3. $\log_2 3$

4.1